Local de ensaladas “Fresh Bowl” – ensaladas personalizadas al gusto

Entrega 2 – BackEnd

Aylin Herrera Soto

21.163.333-6

1. Introducción

El proyecto Fresh Bowl consiste en el desarrollo de un sistema para un local de ensaladas personalizadas, donde los clientes pueden seleccionar ingredientes y realizar pedidos según sus preferencias.

En esta segunda evaluación se desarrolló el backend, encargado de administrar la información del negocio, procesar las operaciones principales y controlar el acceso a los recursos.

El desarrollo utiliza FastAPI, Python y SQLite, incorporando autenticación, gestión de inventario, clientes, pedidos, pagos simulados y consultas mediante GraphQL.

2. Objetivos

2.1 Objetivo general

Desarrollar un backend para Fresh Bowl que permita administrar las operaciones principales del negocio mediante servicios organizados, persistencia de datos y mecanismos de seguridad.

2.2 Objetivos específicos

- Implementar servicios para gestionar ingredientes, clientes, pedidos y pagos.
- Incorporar autenticación y permisos según roles.
- Mantener información persistente mediante una base de datos.
- Implementar un API Gateway para aplicar políticas de acceso.
- Incorporar GraphQL para consultar información de distintos subsistemas.

Realizar pruebas automatizadas para comprobar el funcionamiento del backend.

3. Tecnologías utilizadas

Para el desarrollo se utilizaron las siguientes tecnologías:

- **Python 3.13:** lenguaje de programación.
- **FastAPI:** desarrollo de servicios REST.
- **SQLAlchemy:** conexión y administración de la base de datos.
- **SQLite:** almacenamiento de información.
- **Pytest:** ejecución de pruebas automatizadas.
- **JWT:** autenticación mediante tokens.
- **Argon2:** protección de contraseñas.
- **Strawberry GraphQL:** consultas y agregación de información.
- **HashiCorp Vault:** gestión de secretos, con integración pendiente de verificación.
- **Docker:** preparación del entorno de despliegue.

4. Desarrollo del proyecto

4.1 Servicios y contratos

Se desarrollaron servicios para administrar los principales elementos del negocio: ingredientes, clientes, pedidos, pagos y autenticación.

Cada servicio dispone de operaciones específicas para consultar, registrar o modificar información. También se definieron esquemas para validar los datos recibidos y las respuestas entregadas por la API.

Los contratos permiten establecer qué información necesita cada operación y cómo debe responder el backend.

4.2 Modelo de dominio y UML

El modelo de dominio considera las entidades principales del sistema: usuarios, ingredientes, clientes, pedidos y pagos.

Estas entidades representan la información necesaria para el funcionamiento del negocio y sus relaciones dentro de la base de datos.

Pendiente: incorporar y revisar los diagramas UML definitivos, incluyendo las cardinalidades y la justificación de las relaciones, si todavía no están incluidos en los anexos.

4.3 Sistemas y persistencia

El backend utiliza SQLAlchemy para administrar la comunicación con SQLite.

Se implementaron modelos para almacenar usuarios, ingredientes, clientes, pedidos y pagos, permitiendo conservar la información entre ejecuciones de la aplicación.

El inventario mantiene información sobre el nombre, categoría, precio, disponibilidad y stock de cada ingrediente.

Los pedidos registran los datos del cliente, los ingredientes seleccionados, las cantidades, la modalidad de entrega y el total calculado por el backend.

Limitación: la persistencia mediante volúmenes de Docker todavía no está verificada.

4.4 Stakeholders y seguridad

Se definieron tres roles principales:

- **Cliente:** puede registrarse, iniciar sesión y realizar pedidos.
- **Técnico:** puede consultar información y gestionar determinadas operaciones.
- **Administrador:** dispone de permisos de administración sobre los recursos del sistema.

La autenticación utiliza JWT y las contraseñas se almacenan mediante hashes Argon2, evitando guardar las contraseñas originales.

Las rutas protegidas verifican los permisos correspondientes antes de permitir el acceso.

Limitación: todavía deben fortalecerse los controles para asegurar que los clientes solamente puedan acceder a sus propios pedidos y pagos.

4.5 API Gateway y gestión de secretos

Se implementó un API Gateway mediante middleware de FastAPI, encargado de revisar las políticas de acceso según la ruta, el método HTTP y el rol del usuario.

También se preparó una integración con HashiCorp Vault para gestionar información sensible, como la clave JWT y la dirección de conexión a la base de datos.

La configuración permite utilizar variables de entorno como alternativa cuando Vault no está disponible.

Limitación: el Gateway funciona dentro de la misma aplicación FastAPI, por lo que no existe una separación física de red entre los servicios. Además, la integración completa con Vault todavía no ha sido comprobada.

4.6 Rutas, secretos y GraphQL

Se documentaron las rutas principales del backend y las variables sensibles utilizadas por la aplicación.

Entre ellas se encuentran la clave JWT, la conexión a la base de datos y las credenciales de acceso a Vault.

GraphQL se encuentra disponible mediante el endpoint /api/v1/graphql, protegido para los roles administrador y técnico.

Actualmente se dispone de consultas para obtener ingredientes, pedidos y un resumen general del negocio.

4.7 Clasificación y protección de datos

Los datos se clasificaron según su nivel de sensibilidad: público, interno, confidencial y crítico.

Se identificaron como información crítica las contraseñas, los tokens de autenticación y las credenciales de infraestructura.

El sistema protege las contraseñas mediante Argon2 y utiliza cookies HttpOnly para almacenar el token de autenticación.

Se determinó que no es necesario almacenar información bancaria de los clientes, debido a que el sistema utiliza un proveedor de pagos simulado.

Pendiente: formalizar las políticas de retención y eliminación de datos, y fortalecer medidas adicionales de seguridad antes de utilizar el sistema en producción.

4.8 Agregación GraphQL

Se implementó GraphQL mediante Strawberry para reunir información de los subsistemas Inventory y Orders.

La consulta dashboard permite obtener tres indicadores:

- Cantidad de ingredientes disponibles.
- Cantidad total de pedidos.
- Total de ventas de pedidos no cancelados.

Esto permite consultar información de distintos módulos mediante una única operación.

Limitación: no se han implementado límites de profundidad ni complejidad de consultas, y los permisos se aplican actualmente al endpoint completo, no de manera diferenciada por campo.

4.9 Analytics y gRPC

Se determinó que Analytics sí resulta aplicable al proyecto, ya que permite obtener indicadores generales mediante la consulta dashboard.

Por otra parte, se decidió no implementar gRPC, debido a que los módulos forman parte de una misma aplicación FastAPI y no requieren comunicación entre servicios distribuidos.

Esta decisión evita incorporar complejidad innecesaria en la arquitectura actual.

Pendiente: ejecutar pruebas de carga y registrar mediciones de latencia, rendimiento, errores y consumo de recursos.

4.10 Pruebas, reproducibilidad y despliegue

Se desarrollaron pruebas automatizadas para verificar los módulos de autenticación, inventario, clientes, pedidos, pagos, API Gateway y GraphQL.

En la última ejecución global registrada se obtuvieron 56 pruebas aprobadas y ninguna fallida.

También se prepararon los archivos Dockerfile y Docker Compose para facilitar el despliegue del backend y Vault.

Problema encontrado: durante la construcción de la imagen Docker se presentó un error relacionado con la lectura del Dockerfile por parte de BuildKit. Aunque la configuración de Docker Compose fue validada, el despliegue completo todavía no se ha comprobado.

4.11 Evidencias de pruebas

Para comprobar el funcionamiento del backend se realizaron pruebas sobre los principales módulos del sistema. Estas pruebas permitieron verificar la persistencia de información, el control de acceso, las operaciones de pedidos y pagos, y el funcionamiento de los servicios.

Las evidencias se presentan mediante capturas de pantalla de las ejecuciones realizadas y sus respectivas respuestas.

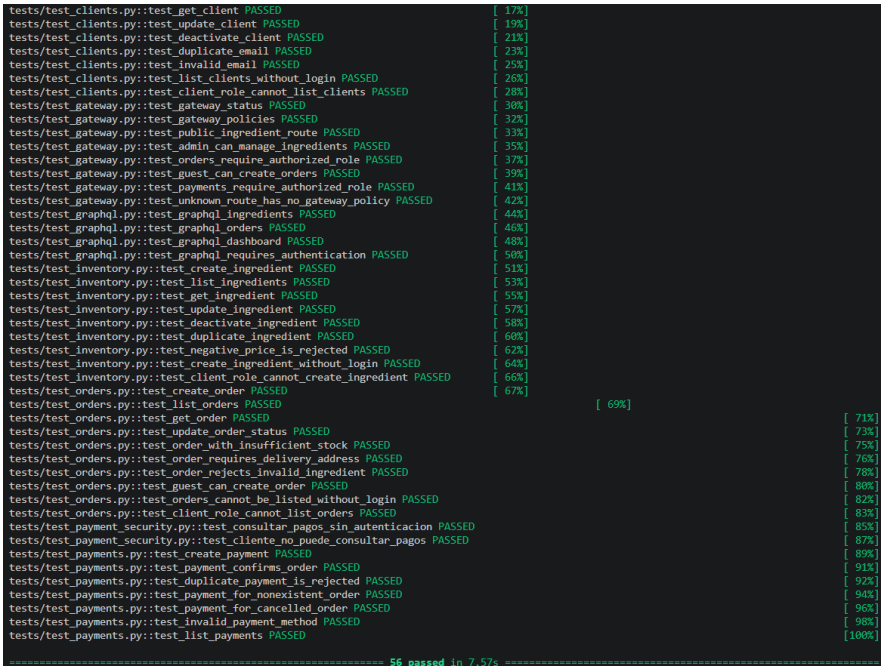


Figura 1. Ejecución de pruebas automatizadas del backend mediante Pytest

Figura 2. Prueba de persistencia

Se implementó persistencia de información mediante SQLite y SQLAlchemy. La comprobación de conservación de datos después de reiniciar el servidor queda pendiente de ejecución.

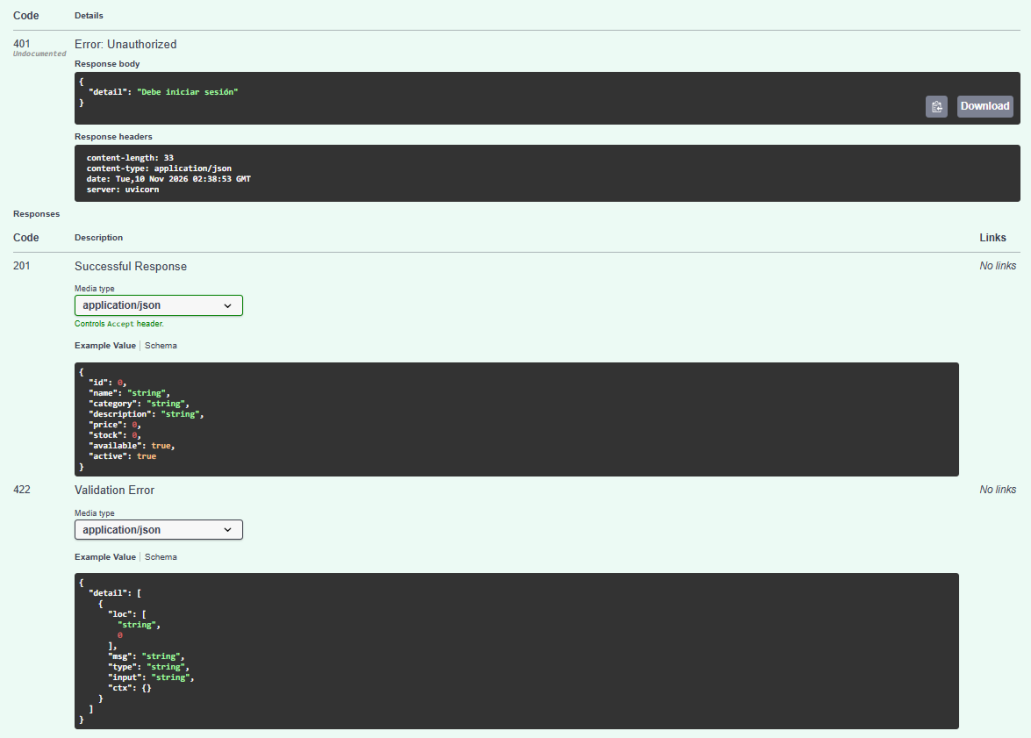


Figura 3. Pruebas de seguridad

Se realizó una prueba de acceso a la operación de creación de ingredientes sin iniciar sesión. El backend respondió con el código HTTP 401 (Unauthorized), rechazando correctamente la solicitud debido a la ausencia de autenticación.

Figura 4. Prueba de pedidos

El backend dispone de operaciones para registrar pedidos y validar los ingredientes seleccionados. El total se calcula utilizando los precios almacenados en el sistema. Queda pendiente incorporar evidencia de ejecución que permita demostrar la creación de pedidos y el rechazo de datos inválidos.

Figura 5. Prueba de pagos

El sistema incorpora un procesamiento de pagos simulado, que permite registrar pagos asociados a pedidos y actualizar su estado. También contempla restricciones para evitar registrar más de un pago por pedido. La validación mediante capturas de ejecución queda pendiente.

El procesamiento se encuentra implementado mediante un proveedor simulado. No se adjunta evidencia de ejecución de esta prueba.

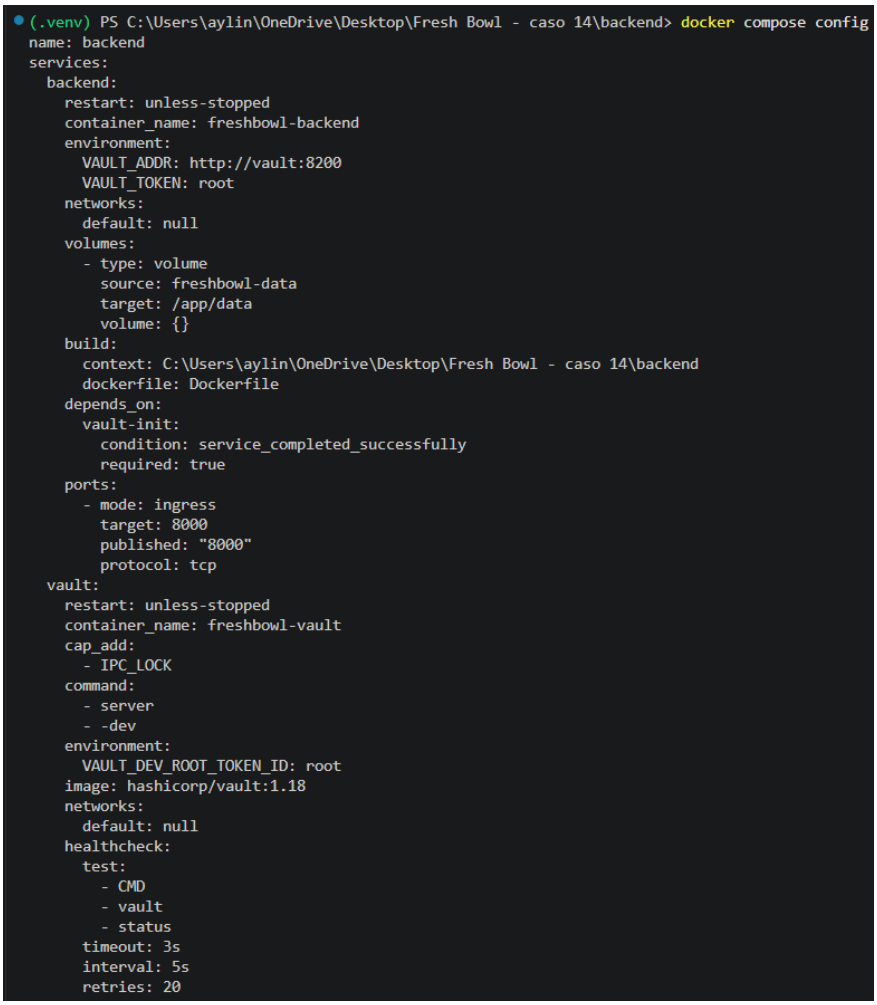


Figura 6. Prueba de despliegue:

5. Resultados obtenidos

El backend cuenta con los siguientes componentes implementados:

- Servicios REST para las operaciones principales.
- Persistencia de datos mediante SQLite y SQLAlchemy.
- Registro, inicio de sesión y autenticación mediante JWT.
- Control de acceso basado en roles.
- Protección de contraseñas mediante Argon2.
- API Gateway implementado como middleware.
- Consultas GraphQL para inventario, pedidos y dashboard.
- Procesamiento de pagos simulado.
- Pruebas automatizadas para los módulos principales.

La última ejecución registrada de Pytest obtuvo 56 pruebas aprobadas y 0 fallidas.

6. Dificultades y limitaciones

Durante el desarrollo se identificaron algunas dificultades y aspectos que no alcanzaron a completarse:

1. **Docker:** la construcción de la imagen presentó problemas al leer el Dockerfile, por lo que el despliegue no quedó verificado.
2. **Vault:** se preparó la integración, pero falta comprobar su funcionamiento completo.
3. **Seguridad:** deben fortalecerse los controles de propiedad de pedidos y pagos.
4. **GraphQL:** faltan límites de profundidad y complejidad de consultas.
5. **Rendimiento:** no se realizaron mediciones de carga que permitan evaluar el comportamiento bajo concurrencia.
6. **Protección de datos:** faltan políticas formales de retención y eliminación.
7. **Integración:** la integración del frontend con el backend se considera parte de la siguiente etapa del proyecto.

7. Conclusión

El desarrollo del backend de Fresh Bowl permitió implementar las principales operaciones necesarias para administrar el negocio, incorporando servicios REST, persistencia, autenticación, autorización y consultas GraphQL.

Las pruebas automatizadas permitieron comprobar el funcionamiento de los distintos módulos, obteniendo 56 pruebas aprobadas en la última ejecución registrada.

Sin embargo, quedaron aspectos pendientes relacionados con el despliegue mediante Docker, la verificación de Vault, la seguridad de algunos recursos y las pruebas de rendimiento.

A pesar de estas limitaciones, se cuenta con una base funcional que permite continuar el desarrollo del proyecto y abordar las mejoras identificadas en las siguientes etapas.